IT HELPDESK TROUBLESHOOTING MANUAL

Network • DNS • Database Connections • Common Application Errors

A practical, beginner-friendly knowledge base for IT support engineers and AI RAG chatbot testing.

How to use this manual: Start with the symptom, run safe diagnostic checks, capture the exact error and time, then escalate with evidence if needed. Commands and examples are illustrative; adapt them to your approved environment.

Safety first: Do not put passwords, access tokens, private keys, or sensitive customer data into tickets or chatbots. Avoid restarting services, changing firewall rules, modifying databases, or deleting data without authorization.

1. Standard troubleshooting workflow

A consistent workflow reduces guesswork and makes incidents easier to resolve.

Step 1 — Define the symptom. Ask what failed, the exact error text, when it began, and whether anything changed recently.

Step 2 — Determine scope. Check whether one user, one device, one application, or many users are affected.

Step 3 — Check basics. Confirm hostname, URL, environment, network/VPN access, credentials status (without requesting the password), and whether the service is expected to be available.

Step 4 — Run read-only diagnostics. Check connectivity, name resolution, service status, logs, and database reachability.

Step 5 — Record evidence. Note timestamps and timezone, host/service, command output, recent changes, and impact. Redact secrets.

Step 6 — Resolve or escalate. Follow the runbook and change process. State what was checked, results, and what team or permission is needed next.

2. Network troubleshooting

Symptoms: a website or server cannot be reached, a connection times out, an application is slow, or a remote service is unreachable.

Check local network configuration

```
ip addr
ip route
```

Check whether a hostname resolves to an address

```
getent hosts server.example.com
```

Test basic reachability (only if ICMP/ping is permitted)

```
ping -c 4 server.example.com
```

Test a TCP port if netcat is installed

```
nc -vz server.example.com 443
```

Test an HTTPS endpoint and show response headers

```
curl -I --max-time 10 https://example.com
```

Interpretation: A timeout can indicate routing, firewall, VPN, host availability, or an overloaded service. “Connection refused” usually means the host was reached but the port is not accepting connections or is actively rejected. Ping failure alone does not prove the host is down; ICMP may be blocked.

Suggested fix: Verify the destination and port, check VPN and routing, confirm the service is listening with the system owner, and ask the network team to validate approved firewall rules. Do not bypass security controls.

3. DNS troubleshooting

DNS translates hostnames (for example, app.example.com) into IP addresses. DNS issues may affect one user, one network, or all clients.

Linux: query the system name service

```
getent hosts app.example.com
```

If dig is installed, query a DNS record

```
dig app.example.com
```

Windows: inspect DNS resolution

```
nslookup app.example.com
```

Windows: show IP configuration and DNS servers

```
ipconfig /all
```

Common symptoms and actions:

“Could not resolve hostname” — check spelling, VPN, DNS server configuration, and whether the record exists.

Wrong or old IP address — verify the authoritative record and TTL with the DNS administrator; caches may temporarily retain earlier values.

Only one machine is affected — compare its network/VPN and DNS settings with a working machine. Use approved cache-refresh steps for that operating system if appropriate.

Important: Do not permanently hard-code an IP address or edit system DNS settings as a workaround without approval.

4. Database connection troubleshooting

Database connection failures can be caused by a wrong host or port, DNS, network policy, database service status, authentication, TLS/certificate issues, connection limits, or incorrect database names.

Start with these checks

1. Confirm the correct environment (development, test, or production), hostname, port, and database name.
2. Verify that the hostname resolves.
3. Test network reachability to the database port using an approved diagnostic tool.
4. Check the database service and server logs with the database administrator.
5. Verify the application is using the expected secret/configuration reference. Never print or share the password.
6. Review recent deployments, firewall changes, certificate rotations, or database maintenance.

Test TCP connectivity (example port; replace with your approved database host/port)

```
nc -vz db.example.com 1433
```

Common ports (defaults can be changed)

Microsoft SQL Server: commonly TCP 1433.
MySQL: commonly TCP 3306.
PostgreSQL: commonly TCP 5432.
Oracle Database: commonly TCP 1521.

Common database errors

Connection timed out — investigate routing, firewall/security rules, VPN, host availability, and whether the correct port is used.

Connection refused — confirm the database is running, listening on the expected interface/port, and accepting TCP connections.

Login failed / authentication failed — check username, account status, authentication method, secret reference, and permissions with the DBA. Do not repeatedly retry if the account may lock.

Unknown database / database does not exist — verify the database name and target environment; do not create or alter production databases without authorization.

TLS or certificate error — check certificate validity, hostname match, trust chain, and client configuration with the platform/DB team. Do not disable certificate validation as a quick fix.

Too many connections / connection pool exhausted — inspect active connections and application pool settings with the owning team. Do not kill sessions or increase limits without impact review.

Example incident note

“Application service cannot connect to db.example.com:1433. DNS resolution succeeds. TCP test times out from the application host at 09:15 IST. No credentials were included in the ticket. Requesting network/DB team to verify route, firewall policy, listener status, and recent changes.”

5. Common application and HTTP errors

HTTP 400 — Bad Request: Request format or required parameters may be invalid. Check request payload and application validation messages.

HTTP 401 — Unauthorized: Authentication is missing or invalid. Check sign-in/session/token configuration securely; never paste tokens into a ticket.

HTTP 403 — Forbidden: The identity may lack permission, or a policy may block access. Ask the resource owner to verify role and access policy.

HTTP 404 — Not Found: Check URL/path, resource ID, deployment route, and whether the resource exists in the correct environment.

HTTP 429 — Too Many Requests: Rate limit may have been reached. Check retry guidance and use backoff rather than sending rapid repeated requests.

HTTP 500 — Internal Server Error: Inspect server-side application logs and recent changes. The response alone usually does not identify the root cause.

HTTP 502 — Bad Gateway: A proxy or gateway received an invalid response from an upstream service. Check upstream health, proxy logs, DNS, and network path.

HTTP 503 — Service Unavailable: The service may be down, overloaded, or under maintenance. Check health status and approved incident communications.

HTTP 504 — Gateway Timeout: An upstream service did not respond in time. Check latency, dependency health, network connectivity, and timeout configuration.

6. Email, access, and account issues

Cannot sign in: Confirm the correct account and login page, check service status, and use the approved password-reset or identity-support process. Never ask a user to send their password.

Access denied to a file or application: Confirm the resource and business need, then route an access request through the approved authorization workflow.

Email not sending: Check connectivity, mailbox quota, recipient address, attachment limits, and any returned non-delivery report. Preserve the exact error and timestamp.

Application is slow: Ask when it started and who is affected; check client/network latency, service health, recent changes, and logs before assuming the database is the cause.

7. Escalation checklist and ticket template

Before escalating, collect the following non-sensitive information:

- User/team and business impact (avoid unnecessary personal information)
- Affected application, host, environment, and service
- Exact error message and timestamp/timezone
- Scope: one user, one site, or many users
- Read-only checks performed and their results
- Recent changes or known incidents
- Steps already attempted and outcome
- Team requested and the specific question to investigate

Sample ticket template

```
Title: [Service] - [short symptom]
Impact: [who/what is affected]
Started: [date/time/timezone]
Environment/host: [non-sensitive identifier]
Error: [exact message; redact secrets]
Checks performed:
- DNS: [result]
- Network/port: [result]
- Service/logs: [result]
Recent changes: [known/unknown]
Request: [specific next check needed]
```

8. Practice questions for your RAG chatbot

Q: What should I check when a website cannot be reached?
A: Verify the URL, network/VPN, DNS resolution, route, destination port, and service health. Use tools such as getent hosts, ping where permitted, nc, and curl; interpret each result carefully.

Q: What does “Could not resolve hostname” mean?
A: The hostname could not be resolved through the system's name-resolution path. Check spelling, VPN/network, DNS configuration, and whether the record exists.

Q: My application cannot connect to SQL Server. What should I check?
A: Confirm the environment, host, port (commonly 1433), DNS, network reachability, listener/service status, authentication configuration, TLS, and database logs. Never expose credentials.

Q: What does HTTP 502 mean?
A: A gateway or proxy received an invalid response from an upstream service. Check upstream health, proxy logs, DNS, and the network path.

Q: What information should I include when escalating?
A: Impact, start time/timezone, affected host/service, exact error, diagnostic checks and results, recent changes, and the specific help needed. Redact secrets.

9. Safety and operational reminders

- Begin with read-only diagnostics.
- Use approved tools and follow organizational runbooks.
- Do not expose passwords, API keys, access tokens, private keys, or confidential data to AI tools.
- Do not disable firewalls, TLS verification, or access controls to make a test pass.
- Avoid production restarts, database writes, permission changes, or data deletion without authorization.
- Validate AI-generated guidance before acting; escalate high-impact or unclear incidents.

End of manual — starter knowledge-base content for an AI IT Helpdesk RAG chatbot.